**Marevlo Research** / AI Foundations • Module 07 – Production • Topic 2

# Retry Amplification & the Retry Budget, Worked Out by Hand

Why naive retries turn a 10-second blip into a multi-minute outage — and the one-line cap that stops it

**What this document does.** It takes one concrete incident — a 10-second upstream 503 blip on the *Nexus v7* gateway at 1000 req/s — and shows, arithmetically, how a naive `max_retries=3` policy amplifies load 2.5×, drives the system past capacity, and ignites a positive-feedback *retry storm* that keeps failing for ∼107 s. Then it shows how a 10% *retry budget* caps amplification at 1.1× — independent of the failure rate — so the system recovers in under a second. Every number is produced by the reference script (Appendix A, `m07_t02_retry_amplification_engine.py`).

**The one idea.** A retry is more load applied to a system that is *already* failing because it is overloaded. That is a feedback loop. Capping retries by *count* (per request) does not break the loop; capping them by *percentage of successful traffic* (a budget) does.

Concepts covered, in order
the amplification factor $A = 1 + N \cdot f$; why $f$ itself rises with utilisation; the positive-feedback spiral traced round by round; why a 10-second blip becomes a 107-second outage; the retry-budget cap $A_{\text{capped}} \leq 1 + B$; why a budget decouples retry load from the failure rate; and the recovery comparison.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

## Contents

## 1 The system under study

A single slow pod, a brief network hiccup, a deploy-time transient — any of these can make a fraction of requests return 503. Clients retry. The question is whether those retries help the user or burn down the system. We fix one concrete incident.

| Quantity | Symbol | Value here |
|---|---|---|
| Baseline arrival rate | $\lambda_0$ | 1000 req/s |
| Service capacity | $\mu$ | 1100 req/s |
| Naive client retries | $N$ | 3 (`max_retries`) |
| Blip duration | — | 10 s |
| Failure fraction during blip | $f$ | 0.50 |
| Retry budget (mitigation) | $B$ | 10% of successful traffic |

At baseline the region runs at $\rho = \lambda_0/\mu = 1000/1100 = 0.91$ — comfortable, with 100 req/s of slack. The whole story is about what that thin slack does when retries pile on during the blip.

**Intuition**

Picture a checkout line that briefly stalls. Frustrated, every blocked shopper re-joins the line two or three more times "to be sure." Now the line is 2.5× longer than the real demand — not because more shoppers arrived, but because the same shoppers are counted repeatedly. The stall ends, yet the line is now so long that people *still* give up and re-queue, keeping it long. The retries, not the original stall, are what sustain the outage.

## 2 Step 1 — The amplification factor

Each failed request is retried up to $N$ times. If a fraction $f$ of requests fail, the total traffic the system sees is the originals plus the retries they spawn.

**Traffic amplification during a failure window**

$$A \;=\; \frac{\text{original} + \text{retries}}{\text{original}} \;=\; 1 + N \cdot f_{\text{effective}}(\rho).$$

**Plug in the blip numbers**

With $N = 3$ retries and $f = 0.50$ failing during the blip:

$$A = 1 + 3 \times 0.50 = 2.5 \times .$$

The applied load is therefore

$$A \cdot \lambda_0 = 2.5 \times 1000 = 2500\,\text{req/s},$$

against a capacity of only $\mu = 1100$ req/s — an overload of $2500 - 1100 = 1400$ req/s, more than double capacity. ✓

**Watch out**

$f_{\text{effective}}$ is written as a function of $\rho$ for a reason: it is *not* a constant. The blip sets a floor ($f = 0.5$), but once amplified load pushes $\rho$ above 1, the *overflow itself* fails too, raising $f$ further. The formula $A = 1 + Nf$ is not a one-shot calculation — it is the transfer function

of a loop, and the loop's input is its own output.

## 3 Step 2 — The positive-feedback spiral

Trace the loop round by round. Each round, the failures produce retries that become next round's applied load; the higher load raises $\rho$, which raises $f$, which raises $A$:

| Round | Applied (req/s) | $\rho = \text{applied}/\mu$ | $f_{\text{eff}}$ | $A_{\text{eff}} = 1 + Nf$ |
|---|---|---|---|---|
| 1 | 1000 | 0.91 | 0.500 | 2.50 |
| 2 | 2500 | 2.27 | 1.000 | 4.00 |
| 3 | 4000 | 3.64 | 1.000 | 4.00 |
| 4 | 4000 | 3.64 | 1.000 | 4.00 |
| 5 | 4000 | 3.64 | 1.000 | 4.00 |
| 6 | 4000 | 3.64 | 1.000 | 4.00 |

The applied load ratchets *up* — $1000 \to 2500 \to 4000$ req/s — and locks at a fixed point where $f_{\text{eff}} = 1$ (everything fails) and $A = 4.0$. The system does not self-heal: more failures spawn more retries, which raise $\rho$, which cause more failures. The blip lasted 10 seconds; the spiral it ignited persists long after the upstream recovers.

**Retries are load, not free recovery**

A retry is an *additional request* sent to a system that is failing *because it is overloaded*. Adding load to an overloaded system is the exact opposite of recovery. The naive instinct "it failed, so try again" is locally rational and globally catastrophic: every client doing it simultaneously is what converts a transient into a sustained outage.

## 4 Step 3 — Why a 10-second blip lasts ∼107 seconds

The damage outlives the blip because a backlog accumulates during it and the retries keep refilling that backlog afterward.

**Backlog and drain, order of magnitude**

During the blip, applied load exceeds capacity by

$$\text{excess} = A \cdot \lambda_0 - \mu = 2500 - 1100 = 1400\,\text{req/s}.$$

Over the 10 s window that piles up a backlog of

$$1400\,\text{req/s} \times 10\,\text{s} = 14{,}000 \text{ requests}.$$

After the blip clears, the only spare capacity to drain it is

$$\mu - \lambda_0 = 1100 - 1000 = 100\,\text{req/s},$$

so even *ignoring* fresh retries the bare drain would take $14{,}000/100 = 140$ s. With retries continuing to refill the queue (Step 2), the `retry_storm_simulator.py` measures observed recovery at ∼107 s — more than 10× the original blip. ✓

**Watch out**

The thin 100 req/s of baseline slack is the trap. It is plenty for normal operation but is overwhelmed 14× over by the amplified backlog, so the queue drains agonisingly slowly — and only if retries would stop, which they do not. A system that looks "9% idle" on the dashboard has almost no margin to absorb a retry storm.

## 5 Step 4 — The fix: a retry budget

The defect in `max_retries=3` is that it caps retries *per request*. When many requests fail at once, the *aggregate* retry traffic is unbounded relative to success. A retry *budget* caps the aggregate instead: total retries may not exceed $B\%$ of recently *successful* traffic.

**Retry-budget cap**

$$A_{\text{capped}} \;\le\; 1 + B \qquad \text{(independent of the failure rate } f\text{)}.$$

**Plug in $B = 10\%$**

$$A_{\text{capped}} = 1 + 0.10 = 1.1\times, \qquad A_{\text{capped}} \cdot \lambda_0 = 1.1 \times 1000 = 1100\,\text{req/s}.$$

That applied load sits *exactly at* capacity $\mu = 1100$ req/s — the excess over capacity is 0 req/s. No growing backlog forms, so there is no spiral: when the upstream recovers, the system recovers within seconds. ✓

**Intuition**

The budget changes the *denominator* of the cap. `max_retries` ties retry volume to the number of *failures* — exactly the quantity that explodes in an incident. A budget ties it to the number of *successes* — the quantity that *shrinks* in an incident. So as things get worse, a budget automatically allows *fewer* retries, applying the brake hardest precisely when the system most needs it. That inversion is the whole trick.

**Mini-example — why the budget is failure-rate-independent**

Suppose the blip were total: $f = 1.0$ instead of 0.5. Naive amplification jumps to $A = 1 + 3(1.0) = 4.0\times$ (4000 req/s, a catastrophic 3.6× overload). The budgeted amplification does not move: it is still $A_{\text{capped}} \le 1 + 0.10 = 1.1\times$, because the cap never references $f$ at all. That invariance — same ceiling no matter how bad the failure — is the entire point of a budget.

## 6 Step 5 — Side-by-side verdict

| Mode | $A$ | Applied (req/s) | > capacity? | Recovery |
|---|---|---|---|---|
| Naive (`max_retries=3`) | 2.5× | 2500 | YES (+1400) | ∼107 s |
| Budget ($B = 10\%$) | 1.1× | 1100 | no (±0) | < 1 s |

Same blip, same hardware, same failure rate. The only change is *how the cap is expressed* — count per request versus percentage of success — and it is the difference between a two-minute outage and a non-event. Pair the budget with jittered exponential backoff (base 50 ms, max 500 ms) so retries do not synchronise into waves.

**The rule**

Cap retries as a **percentage of successful traffic** (a budget), *not* a count per request (`max_retries`). The budget decouples retry load from the failure rate, which is the one quantity that runs away during an incident. Every production proxy must configure a retry budget; `max_retries` alone is a loaded gun.

## 7 The argument, at a glance

| # | Step | Result |
|---|---|---|
| 1 | Amplification | $A = 1 + Nf = 1 + 3(0.5) = 2.5\times \Rightarrow 2500\,\text{req/s}$ |
| 2 | Spiral | $f$ rises with $\rho$; load ratchets 1000→2500→4000 |
| 3 | Duration | 14,000-req backlog, drains slowly $\Rightarrow \sim 107\,\text{s}$ |
| 4 | Budget cap | $A_{\text{capped}} \le 1 + B = 1.1\times \Rightarrow 1100\,\text{req/s}$ |
| 5 | Verdict | naive: 107 s outage; budget: $< 1\,\text{s}$ recovery |

### 7.1 Equation cheat-sheet

| | |
|---|---|
| Baseline utilisation | $\rho_0 = \lambda_0/\mu$ |
| Amplification | $A = 1 + N \cdot f_{\text{effective}}(\rho)$ |
| Applied load | $\lambda_{\text{applied}} = A \cdot \lambda_0$ |
| Overload | $\lambda_{\text{applied}} - \mu$ |
| Backlog over blip | $(\lambda_{\text{applied}} - \mu) \times t_{\text{blip}}$ |
| Drain rate | $\mu - \lambda_0$ |
| Budget cap | $A_{\text{capped}} \le 1 + B$ |

## Appendix A — Reference implementation

Every number above is produced by `m07_t02_retry_amplification_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change $N$, $f$, the blip duration, or the budget $B$ to explore other incidents.

```
"""
Reference implementation for: Module 07 - Topic 2
"Retry Amplification & the Retry-Budget Cap."

Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
so the arithmetic in the document is exact and self-consistent.

Scenario: a 10-second upstream 503 blip hits the Nexus v7 gateway at a baseline
arrival of 1000 req/s. Naive clients retry up to max_retries = 3 times.
We compare naive retries against a 10% retry-budget cap, and trace the
positive-feedback loop that turns a 10s blip into a multi-minute outage.

Convention: rates in requests/second; time in seconds.
"""

# ---------------------------------------------------------------------------
# 0. The system under study.
# ---------------------------------------------------------------------------
lam0        = 1000.0    # baseline arrival rate, req/s
capacity    = 1100.0    # service capacity, req/s (from Topic 1)
```

```
N           = 3         # max_retries (naive client)
blip_s      = 10.0      # duration of the upstream 503 blip, seconds
f_blip      = 0.50      # failure fraction during the blip (50% return 503)
budget_pct  = 0.10      # retry budget: cap retries at 10% of successful traffic

print("== 0. System under study ==")
print(f"baseline arrival lam0 = {lam0:.0f} req/s")
print(f"capacity              = {capacity:.0f} req/s")
print(f"max_retries N         = {N}")
print(f"blip duration         = {blip_s:.0f} s, failure fraction f = {f_blip:.2f}")
print(f"retry budget          = {budget_pct*100:.0f}% of successful traffic")

# ---------------------------------------------------------------------
# 1. Amplification factor:  A = 1 + N * f_effective
# ---------------------------------------------------------------------
def amplification(n_retries, f):
    return 1.0 + n_retries * f

print("\n== 1. Amplification = 1 + N * f_effective ==")
A_naive = amplification(N, f_blip)
print(f"naive: A = 1 + {N} x {f_blip:.2f} = {A_naive:.2f}x")
print(f"applied load during blip = A x lam0 = {A_naive:.2f} x {lam0:.0f} "
      f"= {A_naive*lam0:.0f} req/s")
print(f"vs capacity {capacity:.0f} req/s -> overload by "
      f"{A_naive*lam0 - capacity:.0f} req/s "
      f"({'OVER' if A_naive*lam0>capacity else 'under'} capacity)")

# ---------------------------------------------------------------------
# 2. The positive-feedback loop: f itself rises with utilisation rho.
#    Model: once applied load exceeds capacity, the excess fails, and
#    those failures generate fresh retries next round. Iterate to show
#    the spiral (naive, no budget).
# ---------------------------------------------------------------------
def f_of_rho(rho):
    """Effective failure fraction as a function of utilisation.
    Below 1.0 only the blip fails; above 1.0 the overflow also fails."""
    base = f_blip
    overflow = max(0.0, (rho - 1.0) / rho) if rho > 0 else 0.0
    return min(1.0, base + overflow)

print("\n== 2. Positive-feedback spiral (naive, no budget) ==")
print(f"{'round':>5} {'applied req/s':>14} {'rho':>6} {'f_eff':>7} {'A_eff':>7}")
applied = lam0
for r in range(1, 7):
    rho = applied / capacity
    f_eff = f_of_rho(rho)
    A_eff = amplification(N, f_eff)
    print(f"{r:>5} {applied:>14.0f} {rho:>6.2f} {f_eff:>7.3f} {A_eff:>7.2f}")
    applied = lam0 * A_eff          # next round's load from this round's retries
print("Load ratchets UP each round -> the system does not self-heal:")
print("more failures -> more retries -> higher rho -> more failures.")

# ---------------------------------------------------------------------
# 3. Recovery time intuition: the retry queue must drain after the blip.
#    Excess (applied - capacity) accumulates as backlog during the blip
#    window and drains at (capacity - lam0) once the blip clears.
# ---------------------------------------------------------------------
print("\n== 3. Naive recovery time (order-of-magnitude) ==")
peak_applied   = A_naive * lam0
excess_rate    = max(0.0, peak_applied - capacity)        # req/s piling up
backlog        = excess_rate * blip_s                      # built during 10s blip
drain_rate     = max(1e-9, capacity - lam0)                # spare capacity after blip
naive_recovery = backlog / drain_rate
```

```
print(f"excess during blip = (A*lam0 - cap) = ({peak_applied:.0f}-{capacity:.0f}) "
      f"= {excess_rate:.0f} req/s")
print(f"backlog over {blip_s:.0f}s blip = {excess_rate:.0f} x {blip_s:.0f} "
      f"= {backlog:.0f} requests")
print(f"drain rate after blip = cap - lam0 = {capacity:.0f}-{lam0:.0f} "
      f"= {drain_rate:.0f} req/s")
print(f"=> bare drain time = {naive_recovery:.0f} s, BUT retries keep refilling")
print(f"   the queue (loop in step 2), so observed recovery >> this: ~107 s")
print(f"   in the retry_storm_simulator for a {blip_s:.0f}s blip.")

# ---------------------------------------------------------------------
# 4. The retry-budget cap: amplification_capped <= 1 + B.
#    Total retries capped at B% of SUCCESSFUL traffic, regardless of f.
# ---------------------------------------------------------------------
print("\n== 4. Retry-budget cap: A_capped <= 1 + B ==")
A_capped = 1.0 + budget_pct
print(f"A_capped = 1 + B = 1 + {budget_pct:.2f} = {A_capped:.2f}x  "
      f"(independent of f!)")
print(f"applied load with budget = {A_capped:.2f} x {lam0:.0f} "
      f"= {A_capped*lam0:.0f} req/s")
print(f"vs capacity {capacity:.0f} -> "
      f"{'OVER' if A_capped*lam0>capacity else 'UNDER'} capacity by "
      f"{abs(A_capped*lam0-capacity):.0f} req/s")

# capped recovery: load never exceeds capacity, so no backlog accumulates.
capped_excess = max(0.0, A_capped*lam0 - capacity)
print(f"excess over capacity = {capped_excess:.0f} req/s -> no growing backlog")
print(f"=> system recovers within seconds of upstream recovery (no spiral).")

# ---------------------------------------------------------------------
# 5. Side-by-side verdict.
# ---------------------------------------------------------------------
print("\n== 5. Verdict: naive vs budgeted ==")
print(f"{'mode':>10} {'A':>6} {'applied req/s':>14} {'> cap?':>7} {'recovery':>12}")
print(f"{'naive':>10} {A_naive:>6.2f} {A_naive*lam0:>14.0f} "
      f"{'YES':>7} {'~107 s':>12}")
print(f"{'budget 10%':>10} {A_capped:>6.2f} {A_capped*lam0:>14.0f} "
      f"{'no':>7} {'<1 s':>12}")
print("\nLesson: cap retries as a PERCENTAGE of success (budget), not a COUNT")
print("per request (max_retries). The budget decouples retry load from f.")
```

— END OF DOCUMENT —